Lab 1: Color Spaces & Basic Geometric Transformations

Building image processing fundamentals from scratch using Python





Setting Up Your Image Processing Lab

Build a local environment with Anaconda, VS Code, Python, and OpenCV. This guide takes you from installation to running your first code.

Four Essential Components

Anaconda

Package and environment manager that simplifies installing libraries and creates isolated project environments to prevent conflicts.

Python

The programming language you'll use to write your image processing code.

VS Code

The code editor integrate with development environment where you'll write, run, debug, and manage your Python code.

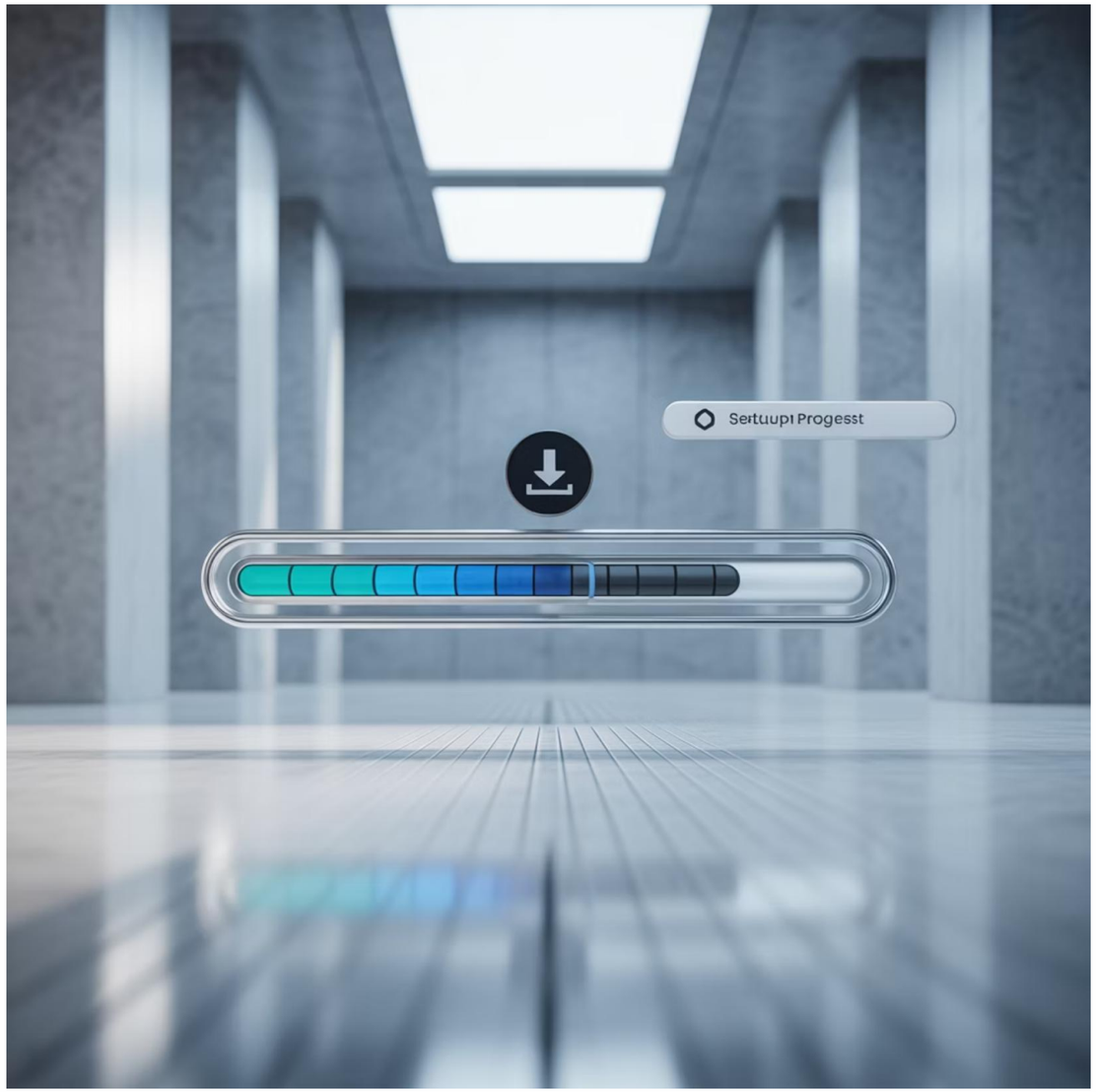
OpenCV

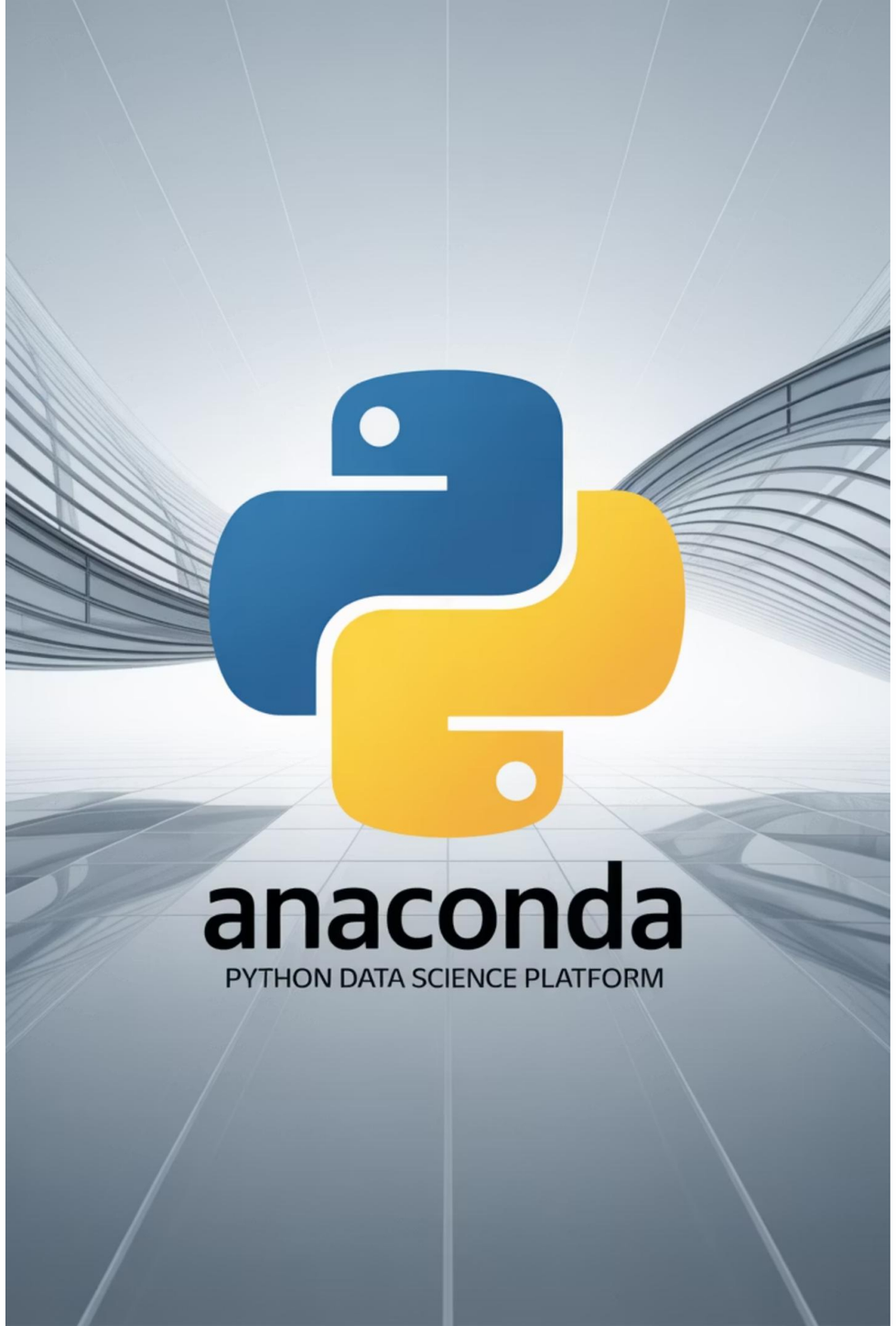
The core computer vision library providing functions for reading, manipulating, analyzing, and displaying images and videos.

Installing Core Tools

Before creating your first project, you need to install two fundamental applications:

- Anaconda and Visual Studio Code.





Step 1: Install Anaconda

01

Download the Installer

Visit the official Anaconda website https://www.anaconda.com/download or **our Moodle** and then download the installer matching your operating system (Windows, macOS, or Linux).

02

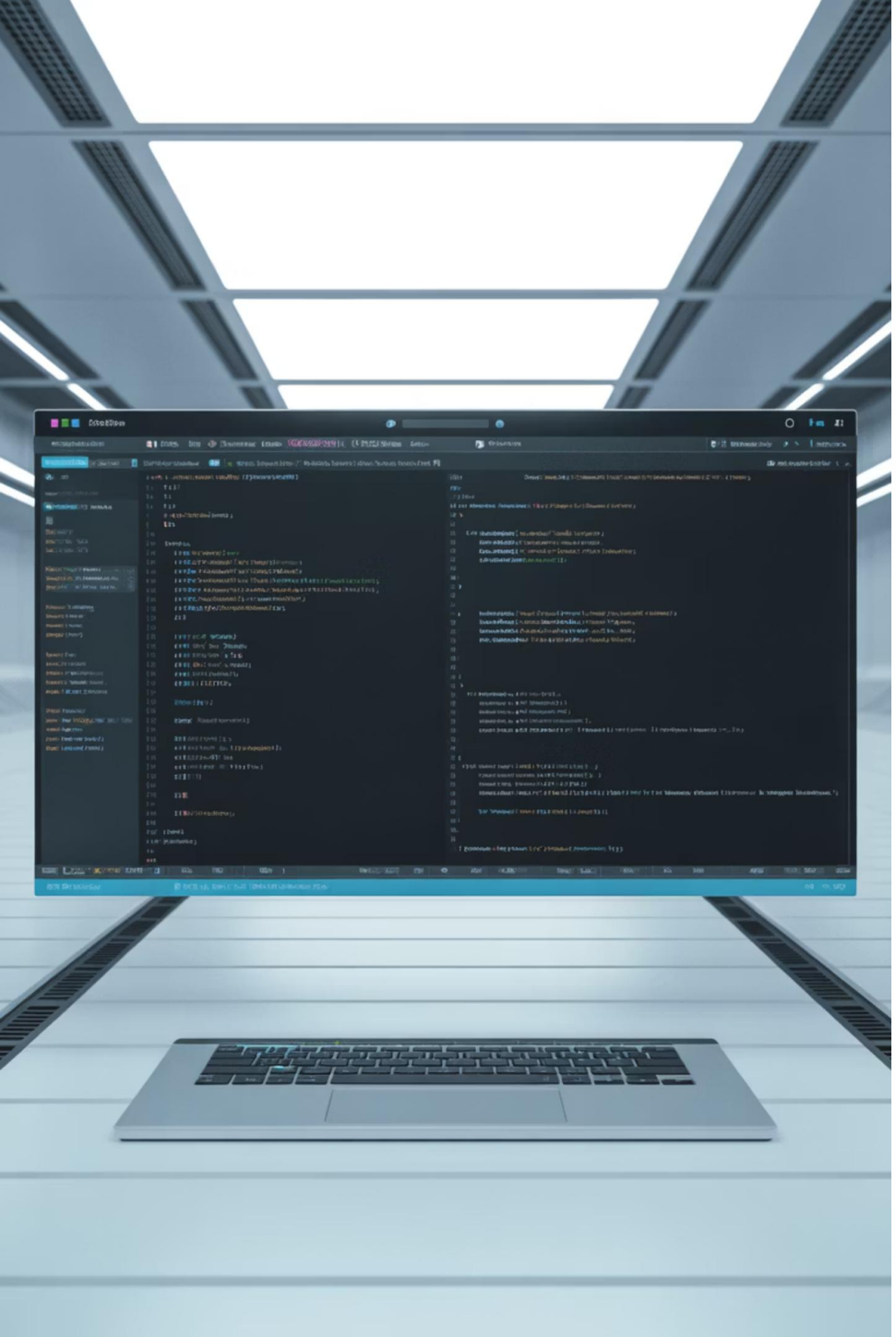
Run the Installation

Execute the downloaded installer and accept the default settings. This installs **Python, Anaconda, and a comprehensive base set of common packages.**

03

Verify Installation

The installation creates shortcuts and adds Anaconda to your system. You'll access it through Anaconda Prompt or your terminal.



Step 2: Install Visual Studio Code

1

Download VS Code

Navigate to the official Visual Studio Code website https://code.visualstudio.com/download or **our Moodle** and then download the installer for your operating system.

2

Complete Installation

Run the installer and ensure the "**Add to PATH**" option remains **checked**. This allows you to open folders in VS Code directly from your terminal or command prompt.

Creating Your Isolated Workspace

Why environments matter: Different projects may require different versions of libraries. Environments let you maintain multiple configurations on the same machine without conflicts.

Creating Your Environment

Execute the following command to create a new environment named `Image-Lab` with Python 3.10:

```
conda create --name Image-Lab python=3.10
```

1 Anaconda processes your request

The system displays all packages that will be installed and calculates dependencies.

2 Confirm installation

Type `y` and press Enter when prompted to proceed with the installation.

3 Wait for completion

Anaconda downloads and installs Python 3.10 and essential packages into your new environment.

Activating Your Environment

```
conda activate Image-Lab
```

After activation, your command prompt changes from `(base)` to `(Image-Lab)`, confirming you're working inside your new environment.

All **subsequent package** installations will be isolated to this environment, keeping your base installation clean.

Installing OpenCV and Dependencies

With your **`Image-Lab`** environment active, install the essential libraries:

```
conda install -c conda-forge opencv numpy matplotlib
```



OpenCV

The primary computer vision library with functions for image processing, object detection, and video analysis.



NumPy

Fundamental package for numerical computing. Images are represented as NumPy arrays for efficient processing.



Matplotlib

Visualization library for creating plots, histograms, and displaying image data during analysis.

The `-c conda-forge` flag instructs Anaconda to use the trusted community channel for the most up-to-date, stable versions. Press `y` to approve the installation.

Connecting VS Code to Your Environment

VS Code needs to know which Python interpreter to use. We'll configure it to use the `Image-Lab` environment you just created, ensuring all your code runs with the correct libraries.



Installing the Python Extension



Open Extensions Panel

In VS Code, click the Extensions icon in the left sidebar (four squares icon) or press Ctrl+Shift+X.



Search for Python

Type "Python" in the search bar. The official Microsoft Python extension should appear first in the results.



Install the Extension

Click the Install button. This extension provides IntelliSense (code suggestion), linting (code error check), debugging, etc.

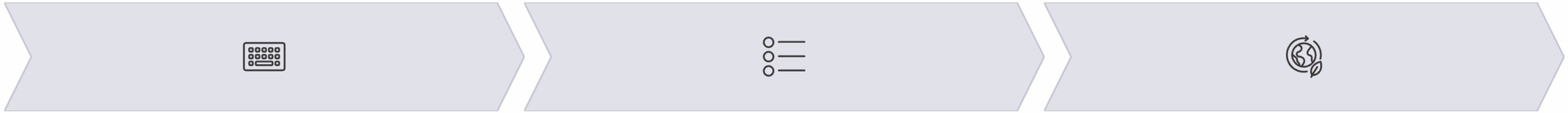
Setting Up Your Project Folder

Create Your Workspace

1. Create a new folder on your computer (e.g., `Image_lab1`)
2. In VS Code, go to **File > Open Folder...**
3. Navigate to and select your newly created folder

Selecting Your Python Interpreter

This crucial step connects VS Code to your `Image-Lab` environment:



Open Command Palette

Press **Ctrl+Shift+P** (Windows/Linux) or **Cmd+Shift+P** (Mac)

Find Python Interpreter

Type `Python: Select Interpreter` and click the matching option

Choose `Image-Lab`

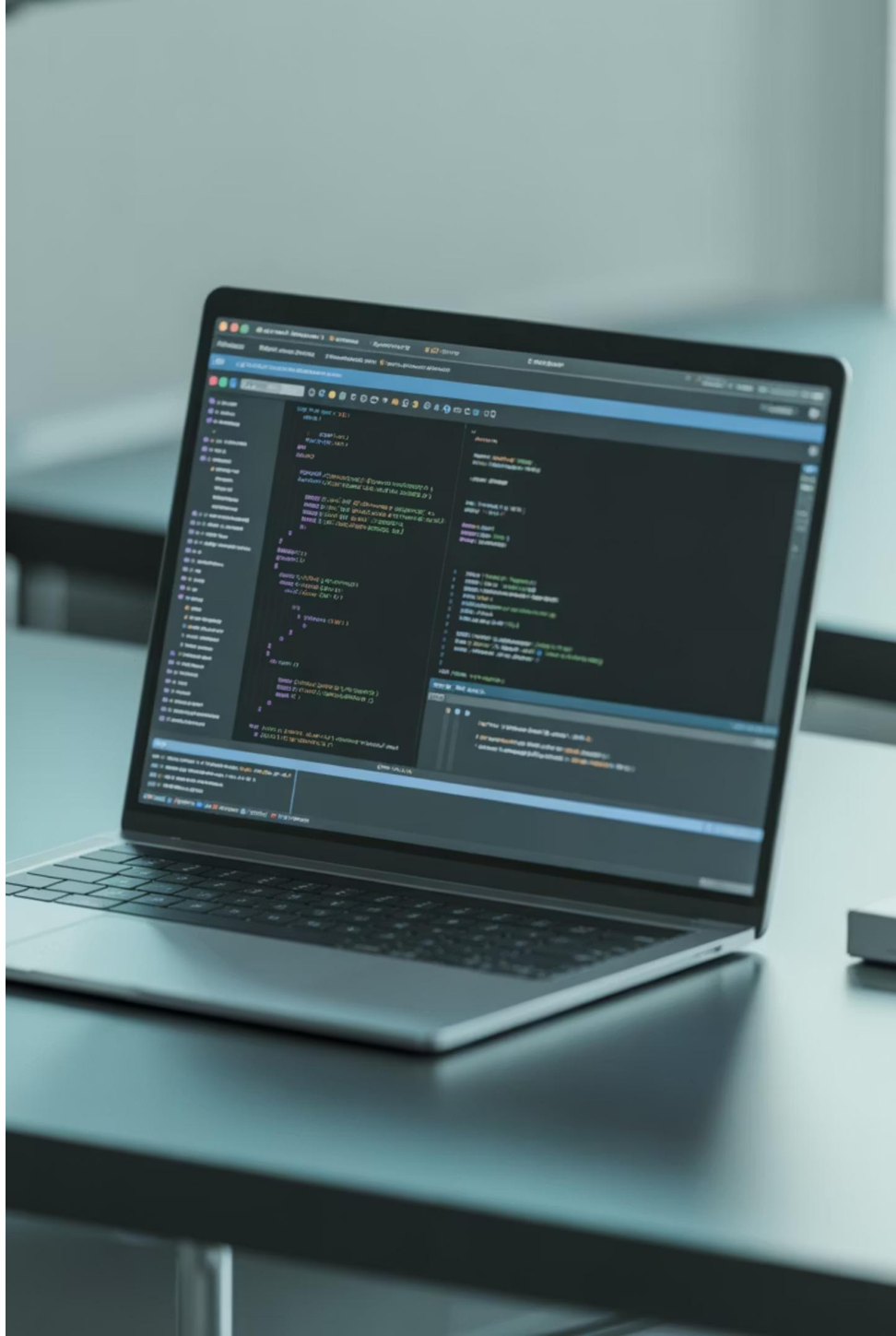
Select the interpreter labeled '`Image-Lab`': **conda** from the list

Verification: Check the bottom-right corner of VS Code. It should display Python 3.10.x ('Image-Lab': conda), confirming successful configuration.

Running Your First code

This is proving everything works by loading and displaying an actual image.

This test verifies your entire setup from environment to libraries.



Preparing Your Test Image

Find an Image

Locate any JPG or PNG image on your computer. Photos of objects, landscapes, or anything.

Copy to Project Folder

Copy your chosen image directly into your project folder (**`Image_lab1`**) that's open in VS Code. For this example, we'll assume it's named `test.jpg`.

Create Python File

In VS Code's sidebar, click the "New File" icon and name it `lab1.py`. This is where you'll write your image processing code.

Copy this complete script into your `lab1.py` file:

```
import cv2
import sys
import os

# Define the image name
image_name = 'test.jpg'  # Make sure this file exists in the current folder

# Load the image
image_path = os.path.join(os.getcwd(), image_name)
print(f"Loading image from: {image_path}")
img = cv2.imread(image_path)

# Check if loaded successfully
if img is None:
    print("Error: Could not read the image.")
    print(f"Check that '{image_name}' is in the folder.")
    sys.exit()

# Display image
cv2.imshow("Original Image", img)
print("Press any key in an image window to close.")

# Wait for key press and clean up
cv2.waitKey(0)
cv2.destroyAllWindows()
```

Understanding the Code

Import Libraries

Load OpenCV, system utilities, and OS path functions needed for file handling and image processing.

Load Image

`cv2.imread()` reads the image file from disk into memory as a NumPy array of pixel values.

Display Results

`cv2.imshow()` creates windows showing both original and processed images for visual comparison.

Clean Up

`cv2.waitKey()` pauses until key press, then `destroyAllWindows()` closes display windows properly.

Running Your Code

Execute Your Code

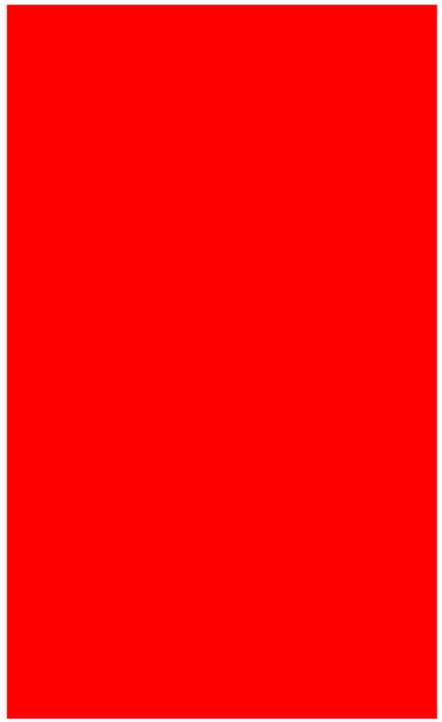
1. Click the **Run** (play button) icon in the top-right corner of VS Code
2. Alternatively, right-click in the editor and select **Run Python File in Terminal**
3. Watch the terminal output for the "Loading image..." message

Task1: Create color images

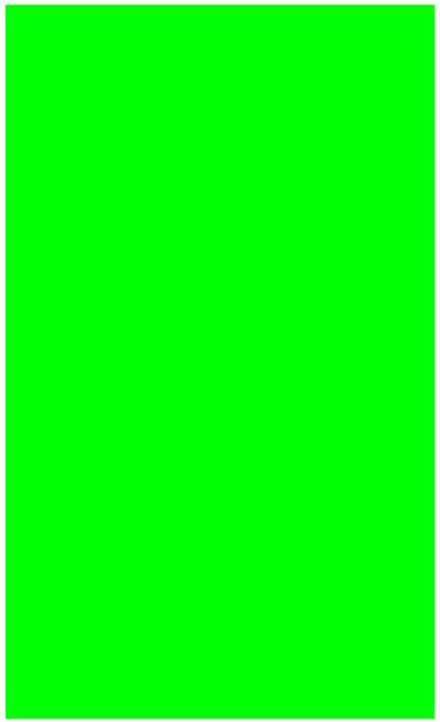
Now that your environment is ready, let's the test with a simple challenge using OpenCV.

Create 3 images, each represents Red image, Green image, and Blue image with 500pixels Height and 300pixels Width.

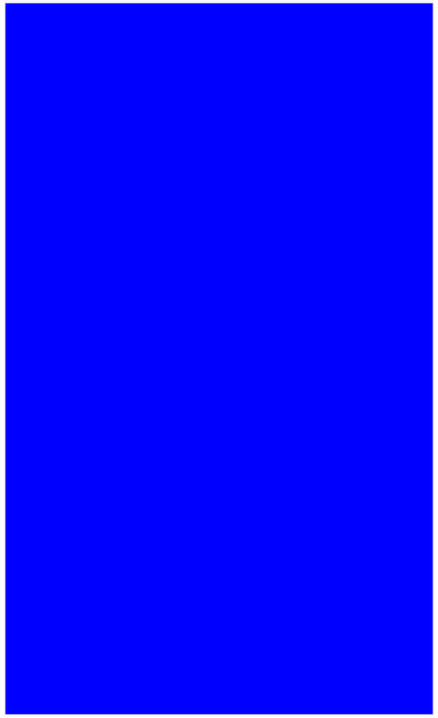
Red Image



Green Image



Blue Image

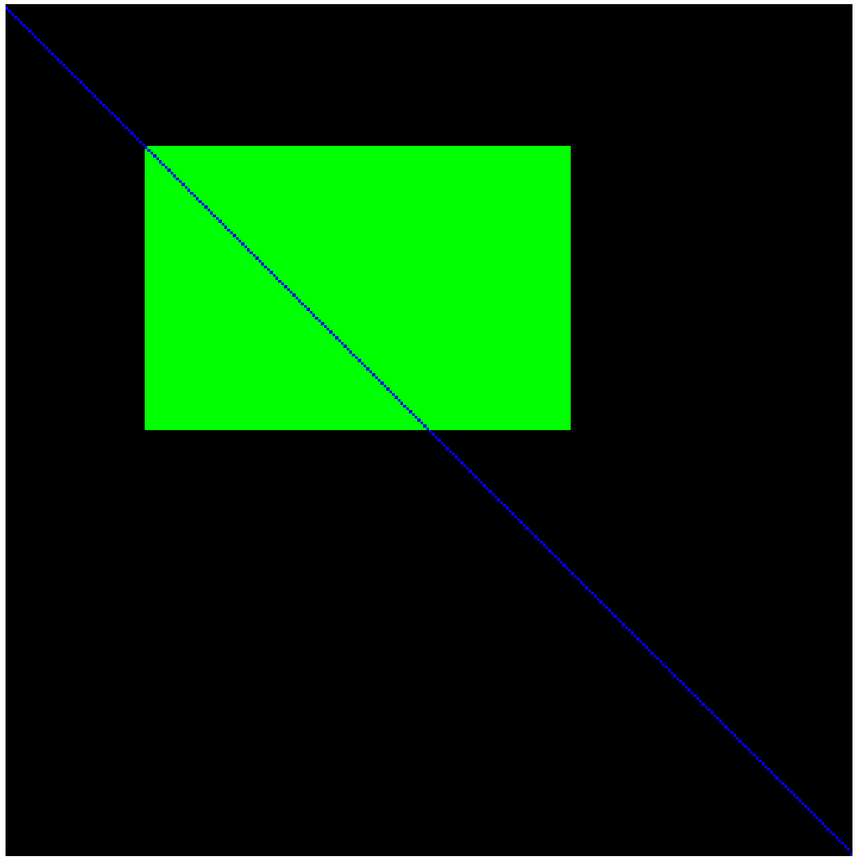


Note that OpenCV use format of BGR image format

Task2: Draw Rectangle and Diagonal

Create a blank image, then draw a rectangle and diagonal of the image.

DO NOT USE build-in: e.g. cv2.line(), cv2.rectangle()



Note that OpenCV use format of BGR image format

Task3: Split Image Channels

Load an image as RGB, then split it to 3 images represent Red, Greed, and Blue, respectively.

DO NOT USE build-in: e.g. cv2.split()

Original Image



Red Image



Green Image



Blue Image



Note that OpenCV use format of BGR image format

Task4: Convert RGB to Grayscale image

Load an image as RGB and then covert the image to Grayscale.

DO NOT USE build-in: e.g. cv2.cvtColor()

Original Image



Grayscale



Note that OpenCV use format of BGR image format

Task5: Convert Grayscale to binary image

Load an image as RGB, covert it to Grayscale and then covert to binary image using your own Threshold.

DO NOT USE build-in: e.g. cv2.cvtColor(), cv2.threshold()

Original Image



Grayscale



Binary



Note that OpenCV use format of BGR image format

Task5: Flip image

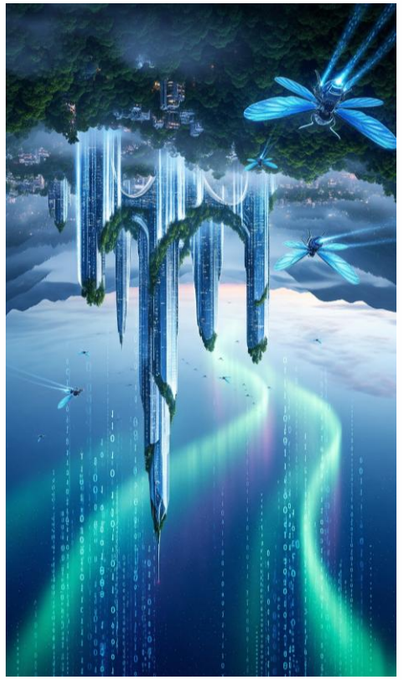
Load an image as RGB, then flip it vertical and horizontal.

DO NOT USE build-in: e.g. cv2.flip()

Original Image



Vertical



Horizontal



Note that OpenCV use format of BGR image format